

Relatório do Trabalho 2: Distância Euclidiana entre Vetores (Otimizada)

Processamento de Alto Desempenho

Henrique Gabriel Rodrigues
Universidade Federal de Pelotas (UFPel)

1 Introdução

Este relatório descreve a implementação e a análise de desempenho do cálculo da distância euclidiana entre um vetor de referência q e um conjunto X com N vetores de tamanho T . Diferentemente da primeira versão do trabalho, esta implementação foi construída inteiramente em C++ puro utilizando a API OpenCL diretamente, sem o intermédio da DSL PolyHok. Além disso, o foco de paralelismo foi direcionado para a CPU (como dispositivo OpenCL) utilizando alocação de memória do tipo SVM (*Shared Virtual Memory*).

O objetivo desta etapa é avaliar o ganho de desempenho obtido e, a fim de enriquecimento, avaliar o impacto da *flag* de compilação `-cl-fast-relaxed-math` (que habilita otimizações agressivas de ponto flutuante em OpenCL) no tempo de execução do *kernel*.

2 Metodologia

2.1 Ambiente e Ferramentas

Os experimentos foram executados em um notebook Lenovo IdeaPad 3, equipado com 8 GB de memória RAM e um processador AMD Ryzen 5 5500U (6 núcleos, 12 *threads*), rodando o sistema operacional Linux Mint 22.3 “Zena”. O código foi escrito em C++ (utilizando os cabeçalhos C++ de OpenCL) e o paralelismo foi direcionado especificamente para a CPU (`CL_DEVICE_TYPE_CPU`).

2.2 Alocação e Escolha do Tamanho de N

A gestão de memória foi realizada através de *Coarse-Grained Shared Virtual Memory* (SVM) via `clSVMAlloc`. Este modelo permite que a CPU (hospedeira) e o dispositivo OpenCL (neste caso, a própria CPU operando via *runtime* OpenCL) compartilhem os mesmos ponteiros virtuais, evitando cópias explícitas através de `clEnqueueWriteBuffer`. O tamanho do conjunto, mantendo consistência com a avaliação anterior, foi mantido em $N = 3.584$.

2.3 Coleta de Tempos

A medição de tempo incluiu somente o lançamento e a execução do *kernel*. Foi utilizada a biblioteca `std::chrono`, nativa do C++, para medir o tempo de execução. As operações

de alocação de buffers SVM, inicialização de vetores, mapeamento/desmapeamento de memória e compilação JIT do *kernel* foram excluídas da região de medição.

2.4 Experimentos e Métricas

A título de curiosidade e comparação, o programa possui duas variações de compilação do *kernel* OpenCL:

1. **Normal:** Compilação padrão do driver OpenCL, sem nenhuma *flag* de otimização.
2. **Fast Math:** Compilação utilizando a *flag* `-cl-fast-relaxed-math`. Essa *flag* habilita várias otimizações que priorizam o desempenho máximo de cálculos em ponto flutuante em detrimento da precisão matemática estrita ao padrão IEEE 754.

Para cada programa e cada tamanho $T \in \{32, 64, 128, 256, 512, 1024, 2048, 4096\}$, o programa foi executado 30 vezes. Com base nessa amostragem, calculou-se a média, mediana e o intervalo de confiança (IC de 95%). Adicionalmente, calculou-se as seguintes taxas de processamento e realizou-se uma série de análises estatísticas para avaliar o impacto da otimização matemática:

- **P-Value (Teste de Mann-Whitney):** para avaliar a significância estatística das diferenças de tempo.
- **Cohen's d:** para medir o tamanho do efeito (a magnitude da diferença).
- **Rank Biserial Correlation:** uma variante não-paramétrica do Cohen's d que também avalia o tamanho do efeito.

3 Resultados e Análise

As tabelas a seguir apresentam os resultados estatísticos e as métricas de desempenho para as execuções Normal e Otimizada (*Fast*).

Tabela 1: Média, desvio, mediana e IC para $N = 3.584$

T	Normal			Fast-Math		
	Média $\pm$ Desvio (ms)	Mediana (ms)	IC 95%	Média $\pm$ Desvio (ms)	Mediana (ms)	IC 95%
32	0,505 $\pm$ 0,105	0,503	[0,466; 0,545]	0,492 $\pm$ 0,098	0,505	[0,456; 0,529]
64	0,629 $\pm$ 0,106	0,597	[0,590; 0,669]	0,634 $\pm$ 0,113	0,607	[0,592; 0,676]
128	0,726 $\pm$ 0,109	0,720	[0,685; 0,766]	0,711 $\pm$ 0,085	0,717	[0,679; 0,743]
256	0,968 $\pm$ 0,101	0,958	[0,930; 1,006]	0,961 $\pm$ 0,119	0,942	[0,916; 1,005]
512	1,372 $\pm$ 0,119	1,393	[1,327; 1,416]	1,404 $\pm$ 0,089	1,398	[1,371; 1,437]
1024	2,292 $\pm$ 0,171	2,267	[2,228; 2,355]	2,333 $\pm$ 0,184	2,268	[2,264; 2,401]
2048	3,950 $\pm$ 0,468	3,928	[3,775; 4,124]	3,919 $\pm$ 0,165	3,909	[3,858; 3,981]
4096	7,296 $\pm$ 0,205	7,304	[7,219; 7,372]	7,389 $\pm$ 0,432	7,296	[7,228; 7,551]

Tabela 2: Taxas de Processamento e Análise Estatística

T	Normal		Fast-Math		P-Value	Cohen's d	Rank Bis.	Rel. Estatística?
	P_v (vet/s)	P_e (elem/s)	P_v (vet/s)	P_e (elem/s)				
32	7.090.342	226.890.954	7.281.980	233.023.390	0,614	0,131	-0,029	Não
64	5.695.151	364.489.692	5.654.300	361.875.246	0,873	-0,041	-0,007	Não
128	4.939.805	632.295.108	5.043.079	645.514.156	0,558	0,152	-0,120	Não
256	3.702.558	947.854.947	3.731.285	955.209.195	0,795	0,067	-0,076	Não
512	2.613.168	1.337.942.380	2.552.765	1.307.015.854	0,235	-0,310	0,134	Não
1024	1.564.016	1.601.552.844	1.536.487	1.573.363.205	0,374	-0,231	0,136	Não
2048	907.447	1.858.452.063	914.498	1.872.892.080	0,738	0,087	0,016	Não
4096	491.253	2.012.175.803	485.017	1.986.630.629	0,287	-0,277	0,096	Não

3.1 Discussão

Os testes estatísticos aplicados nas amostras de 30 execuções demonstram claramente que **não houve melhora estatisticamente significativa** ao utilizar o modelo de matemática relaxada. Para todos os tamanhos de T , o P -Value é superior a 0,05 (o padrão usual para rejeitar a hipótese nula), indicando que a diferença entre as médias é meramente fruto de ruídos e flutuações naturais dos tempos de processamento. As métricas de tamanho de efeito (*Cohen's d*) e correlação (*Rank Biserial*) também indicam efeitos pequenos, nulos, ou até mesmo apontando para um levíssimo aumento de tempo em alguns cenários (mero fruto da variabilidade do hardware).

A hipótese mais provável que explica esse comportamento é o fato de que o cálculo de distância euclidiana não possui intensidade aritmética (não é *compute-bound*) o suficiente para que relaxar a precisão de cálculos em ponto flutuante resulte em ganho no *throughput*. A fórmula exige multiplicações e subtrações simples com apenas uma operação mais complexa (*sqrt*) ao final. Essa baixa intensidade aritmética por elemento faz com que o gargalo do processamento não reside na Unidade Lógica e Aritmética (ALU).

O desempenho (*throughput*) medido em elementos processados por segundo (P_e) exibe uma curva de eficiência ascendente, com um crescimento menor nos valores mais altos. O tempo de execução da CPU torna-se rapidamente delimitado pela largura de banda da memória, tornando-se *memory-bound*). A latência de acesso à RAM para varrer os vetores acaba ofuscando qualquer micro-otimização em nível de compilador para instruções de ponto flutuante. Isso significa que nas entradas maiores a CPU passa a maior parte do tempo ociosa, esperando os dados da RAM.

4 Conclusão

A versão desenvolvida em C++ puro rodando na CPU demonstrou que o limite de desempenho da aplicação é imposto fortemente pela latência e largura de banda da memória (*memory-bound*), e não pela complexidade computacional da função avaliada em si (a mesma conclusão obtida ao avaliar o programa em PolyHok que executava na GPU). Consequentemente, o uso da diretiva de otimização matemática agressiva (*-cl-fast-relaxed-math*) revelou-se ineficaz para melhorar as taxas de processamento P_e e P_v , comprovado pela ausência de relevância estatística, independentemente do tamanho de T .